

System Design Mastery Roadmap

For a CS/ML engineer with strong fundamentals, new to designing systems at scale

Format per stage: What to learn · Why · Hands-on exercise/diagram · Tools · Project · Must be able to explain before moving on · Prerequisites · Real-world transfer · Difficulty · Time estimate.

How teaching actually happens: This doc is the map, not the lesson. In conversation, I ask you to reason first, push back on hand-wavy answers, and only confirm/correct after you've committed to a position. Each stage ends with a comprehension check — if you can't explain the "why" in your own words, we don't move on.

STAGE 0 — Mental Model & Setup (Difficulty: ★☆☆☆☆, ~1 day)

What to learn: How to think about a "system" as: clients → network → stateless compute → stateful storage → (async: queues/workers). Every topic in this roadmap is really answering one of two questions: *"where does this piece live"* and *"what happens when it fails or gets overloaded."* **Why:** Without this frame, system design becomes a list of buzzwords (Kafka, Redis, sharding) with no way to decide which one applies to a given problem. **Deliverable:** Sketch (by hand or draw.io/Excalidraw) a simple 3-tier system for something you already know — the FastAPI ML service you deployed. Label client, load balancer, app servers, database, cache. This becomes your baseline diagram vocabulary for everything after. **Prerequisites:** None — this is the on-ramp.

STAGE 1 — Core Building Blocks (Difficulty: ★★☆☆☆, ~1 week)

What to learn:

- Vertical vs. horizontal scaling, and why horizontal eventually wins
- Latency vs. throughput — and why they trade off against each other
- The latency numbers every engineer should know (L1 cache → RAM → SSD → network round trip → cross-region) — orders of magnitude, not exact numbers
- CAP theorem, and why "pick 2 of 3" is a soundbite that undersells PACELC

- Reliability vs. availability vs. fault tolerance, "nines" math (99.9% vs 99.99% downtime/year)

Why: Every tradeoff decision later in the roadmap (SQL vs NoSQL, sync vs async, sharding strategy) collapses back to these primitives. If you don't have this vocabulary locked in, every later topic becomes memorization instead of derivation.

Exercise (build, not read):

1. Given a target of 50,000 requests/sec at ~10ms/request, calculate how many single-threaded workers you'd need — then explain why that number is a floor, not a real capacity plan.
2. Take a real outage scenario (single DB write node dies) and reason through what CAP forces you to choose in that moment.

Tools: None yet — this is pure reasoning, pen and paper / whiteboard.

Must explain before moving on:

- Why horizontal scaling requires statelessness (or externalized state) to actually work
- A concrete example of when you'd trade latency for throughput, and vice versa
- What CAP actually gives up when you choose availability during a partition

Prerequisites: Stage 0. **Real-world transfer:** This is the vocabulary every interviewer expects you to use unprompted when justifying a design choice — "I'm choosing eventual consistency here because..." is the sentence structure that signals seniority.

STAGE 2 — Networking (Difficulty: ★★☆☆☆, ~1 week)

What to learn:

- DNS resolution path (browser → resolver → root → TLD → authoritative)
- Load balancers: L4 vs. L7, routing algorithms (round robin, least connections, consistent hashing), health checks
- CDNs: push vs. pull caching, edge invalidation
- REST vs. GraphQL vs. gRPC vs. WebSockets — what problem each one specifically solves
- Synchronous vs. asynchronous communication, and why "just make it async" is sometimes a trap that hides a queueing/backpressure problem instead of solving it

Why: This is the layer where "the request" actually travels. You can't reason about latency budgets, failure domains, or where to add redundancy without knowing what's between the client and your code.

Exercise: Trace an actual request path for your FastAPI service (as if it were public and behind a CDN + load balancer) end to end, and mark where each hop adds latency and where each hop is a single point of failure today.

Tools: nginx or Traefik (spin up a toy L7 load balancer locally with Docker), `dig/curl -v` to inspect DNS/HTTP behavior directly.

Project: Stand up two instances of a simple FastAPI app in Docker, put nginx in front as a load balancer, kill one instance, and observe/explain the failover behavior.

Must explain before moving on:

- When you'd pick gRPC over REST (and the cost of that choice — human readability, browser support)
- Why a CDN helps latency but does nothing for a write-heavy API
- What "sync call chain of 5 services" does to your p99 latency, concretely

Prerequisites: Stage 1 (latency intuition). **Real-world transfer:** Almost every system design interview starts with client → LB → service, so this has to be automatic, not derived live.

STAGE 3 — Data Storage (Difficulty: ★★★★★☆, ~2 weeks — this is the core, don't rush it)

What to learn:

- SQL vs. NoSQL — reframed as "what access pattern and consistency need forces which," not a popularity contest
- Indexing: B-tree vs. hash index, why an index speeds reads but costs writes
- Replication: leader-follower, multi-leader, leaderless — and replication lag as a real failure mode, not a footnote
- Sharding/partitioning: range-based, hash-based, directory-based — and hot-spot failure modes for each
- ACID vs. BASE, isolation levels (read committed → serializable), why "eventually consistent" is a spectrum, not an on/off switch

Why: This is where most system design interviews are actually won or lost — almost every case study reduces to "how do you store and access this data at scale." Your ML background already gives you database familiarity (feature stores, training data pipelines); this stage generalizes that into the storage-tradeoff mental model used for any system.

Exercise (build from scratch style, borrowed from your ML habit):

1. Design a hash-based sharding scheme for a table of 500M user rows across 8 shards, by hand. Then explain what happens when you need to go from 8 shards to 12 — and why consistent hashing exists specifically to fix that pain.
2. Walk through what breaks if a leader-follower DB's follower is 3 seconds behind and a user writes then immediately reads their own write.

Tools: PostgreSQL + a NoSQL store (MongoDB or DynamoDB local) run locally in Docker; **EXPLAIN ANALYZE** to actually see an index change a query plan.

Project: Take a dataset you already know (from your ML work) and design two schemas for it — one optimized for a normalized relational read pattern, one optimized for a denormalized NoSQL read pattern — and justify the difference by the query pattern, not by preference.

Must explain before moving on:

- A concrete scenario where NoSQL is correct and SQL would actively hurt you, and the reverse
- Why replication solves availability/read-scaling but not write-scaling
- Why sharding is a last resort, not a first move (and what you'd try before reaching for it)

Prerequisites: Stage 1 (CAP/reliability), basic SQL (you already have this). **Real-world transfer:** Data storage tradeoffs are the single most-tested area in system design interviews across FAANG-style loops.

STAGE 4 — Caching (Difficulty: ★★☆☆☆, ~4–5 days)

What to learn:

- Cache patterns: cache-aside, write-through, write-back, read-through
- Eviction policies: LRU, LFU, TTL — and when each is the wrong choice
- Invalidation strategies — "there are only two hard things in computer science: cache invalidation and naming things"
- Where to cache: client, CDN, application layer, database layer — and multi-layer cache coherence problems

Why: Caching is the highest-leverage, lowest-cost lever for latency and load reduction — and also the single most common source of subtle correctness bugs (stale reads, thundering herd).

Exercise: Design the cache-invalidation strategy for a product-price cache that must never show a stale price after an admin updates it — and explain the tradeoff between "invalidate immediately" and "short TTL, accept some staleness."

Tools: Redis, run locally in Docker; implement a basic LRU cache in Python by hand (no library) to make the eviction mechanics concrete — this mirrors your ML "implement it from scratch first" habit well.

Project: Add a Redis cache-aside layer in front of a slow endpoint in your FastAPI service, benchmark p99 latency before/after, then intentionally break cache invalidation and observe the stale-data bug you just created.

Must explain before moving on:

- Why write-through is safer but slower than write-back, and when that tradeoff is worth it
- What "thundering herd" is and one concrete mitigation (jittered TTLs, request coalescing)

Prerequisites: Stage 2 (networking, for CDN caching), Stage 3 (storage, for cache-vs-source-of-truth reasoning). **Real-world transfer:** Caching shows up in essentially every case study as the first lever an interviewer expects you to reach for once you've identified a hot read path.

STAGE 5 — Distributed Systems (Difficulty: ★★★★★, ~2 weeks)

What to learn:

- Consistency models: strong, eventual, causal — mapped explicitly back to CAP/PACELC from Stage 1
- Consensus, at an intuition level: why you need it (leader election, distributed locks), what Raft/Paxos actually guarantee — not proofs, but what problem they solve and why it's hard
- Message queues: Kafka vs. RabbitMQ vs. SQS — ordering guarantees, delivery semantics, backpressure
- Pub/sub and fan-out patterns, event-driven architecture

- Idempotency and "exactly-once delivery" — why it's effectively a myth, and what you do instead (idempotency keys, dedup on the consumer)

Why: This is where ML-adjacent intuition (you already think about race conditions in async training pipelines and eventual consistency in distributed data loaders) starts paying off directly. This stage is also where most candidates' reasoning becomes hand-wavy under interview pressure — precision here is a real differentiator.

Exercise: Design a "at-least-once delivery + idempotent consumer" flow for a payment processing queue, and explain exactly where duplicates can be introduced and how your design neutralizes them.

Tools: Kafka (or Redpanda, lighter to run locally) in Docker; publish/consume a toy event stream; deliberately kill a consumer mid-processing and observe redelivery behavior.

Project: Build a minimal producer/consumer pipeline (FastAPI producer → Kafka topic → Python consumer that writes to Postgres) and add an idempotency key so re-delivered messages don't double-write.

Must explain before moving on:

- The difference between "the message was delivered" and "the effect was applied exactly once," and why systems conflate these
- When you'd choose Kafka over RabbitMQ (throughput/replay/ordering-by-partition) vs. the reverse (simpler routing, lower ops overhead)

Prerequisites: Stages 1, 3. **Real-world transfer:** This is the material that separates "junior system design answer" from "senior system design answer" — interviewers specifically probe here.

STAGE 6 — API Design & Service Architecture (Difficulty:

★★★★☆☆, ~1 week)

What to learn:

- Rate limiting algorithms: token bucket, leaky bucket, fixed/sliding window — mechanics and failure modes of each
- API gateways — what they centralize (auth, rate limiting, routing) and why that's better than scattering it per-service

- Microservices vs. monoliths — real operational tradeoffs (deploy independence vs. distributed- systems tax), not tribal allegiance
- Service discovery, API versioning, backward compatibility

Why: This stage is "how do independently-deployed pieces of your system talk to each other and stay compatible over time" — the organizational/operational layer on top of the technical layers above.

Exercise: Implement a token bucket rate limiter from scratch in Python (no library), then explain why it handles bursts better than a fixed window counter.

Tools: Redis (for a distributed rate limiter using **INCR** + TTL), FastAPI middleware.

Project: Add real rate limiting to your FastAPI service using Redis so it works correctly across multiple app instances, not just in-process.

Must explain before moving on:

- Why a naive in-memory rate limiter breaks the moment you have more than one server
- One concrete cost of microservices that a monolith doesn't have (distributed transactions, network calls replacing function calls)

Prerequisites: Stages 2, 4. **Real-world transfer:** Rate limiter design is itself a standalone, extremely common interview question — this stage doubles as prep for that specific case study.

STAGE 7 — Observability (Difficulty: ★★☆☆☆, ~4–5 days)

What to learn:

- Structured logging, log levels, log aggregation
- Metrics: the RED method (Rate, Errors, Duration) and USE method (Utilization, Saturation, Errors), and why high-cardinality metrics blow up your monitoring bill
- Distributed tracing: spans, context propagation across service boundaries
- Alerting philosophy: symptom-based ("latency is high") vs. cause-based ("CPU is high") alerts, and why alert fatigue kills on-call teams

Why: A system you can't observe is a system you can't operate — this is the stage that answers "how do you know it's broken, and where."

Exercise: Given a real p99 latency spike, sketch the trace-through you'd want to see across 3 services to localize the bottleneck.

Tools: Prometheus + Grafana (Docker Compose stack), structured logging with `structlog` in your FastAPI app.

Project: Instrument your FastAPI service with Prometheus metrics (request count, latency histogram, error rate) and build one Grafana dashboard from them.

Must explain before moving on:

- Why you'd alert on error rate/latency (symptom) rather than "CPU > 80%" (cause) as your primary page
- What a trace gives you that logs and metrics individually don't

Prerequisites: Stage 2. **Real-world transfer:** Directly reusable for your ML deployment work — model-serving observability (latency, error rate, and later drift) is this exact stack applied to inference traffic.

STAGE 8 — DevOps / MLOps (Difficulty: ★★★★★☆, ~2–3 weeks)

What to learn:

- CI/CD: what "continuous" actually buys you beyond automation (fast feedback, small diffs, reversibility)
- Infrastructure as Code: declarative vs. imperative, config drift
- Container orchestration: Kubernetes core objects and *why each exists* — Pod → Deployment → Service → Ingress, and autoscaling (HPA)
- ML-specific:
 - Model serving at scale: batching requests, model servers (Triton/TorchServe), GPU utilization, autoscaling on queue depth vs. CPU
 - Feature stores: online/offline consistency, point-in-time correctness (avoiding training/ serving skew — you've already brushed this via data leakage in your ML roadmap)
 - Drift detection: data drift vs. concept drift, monitoring strategy
 - A/B testing, shadow deployment, canary releases for models
 - Online vs. offline inference — batch pipeline vs. real-time serving tradeoffs

Why: This is where your existing Docker/FastAPI/cloud deployment experience gets pushed from "single instance" to "at scale, with orchestration and ML-specific failure modes." It directly extends Stage 9 of your ML roadmap.

Exercise: Design the retraining + rollout pipeline for a model that degrades under data drift: what triggers retraining, how do you validate the new model before it's live, how do you roll it back if it's worse.

Tools: Docker Compose → minikube/kind for local Kubernetes, GitHub Actions for CI/CD, MLflow for model registry/versioning (you may already have touched this).

Project: Take the FastAPI model-serving app from your ML roadmap's Stage 9 and: (1) add a CI pipeline that runs tests + builds the image on push, (2) deploy it to a local Kubernetes cluster with a Deployment + Service + HPA, (3) add a canary rollout step where 10% of traffic hits a new model version before full rollout.

Must explain before moving on:

- Why a Kubernetes Deployment exists as a separate object from a Pod (self-healing, rolling updates)
- The difference between data drift and concept drift, with a concrete example of each
- Why online and offline feature computation diverging is a silent, dangerous bug — not just an inconvenience

Prerequisites: Stages 1, 2, 5, 7; your existing Docker/FastAPI/cloud deployment experience.

Real-world transfer: This stage is the direct bridge between "I can train and serve one model" and "I can be trusted to run ML in production at a company with real traffic."

STAGE 9 — Interview Methodology (Difficulty: ★★★★★, ongoing — revisit before every case study)

What to learn:

1. Requirements clarification — functional requirements, non-functional requirements (latency, availability, consistency needs), explicit scope boundaries
2. Back-of-envelope estimation — QPS, storage growth, bandwidth, from stated user/traffic numbers
3. High-level design sketch — identify components and data flow before any deep dive
4. Deep dive — pick 1–2 components, justify tradeoffs under follow-up questioning
5. Bottleneck/failure-mode identification — what breaks first as scale increases, and how you'd detect and mitigate it

Why: Knowing all the building blocks doesn't help if you can't structure 35–45 minutes of reasoning under pressure. This is the meta-skill that turns Stages 1–8 into an actual interview performance.

Exercise: Given only "design a system that lets users share short links," write down every clarifying question you'd ask before drawing anything — then compare against what an experienced interviewer would actually want asked.

Must explain before moving on:

- Why skipping requirements clarification is the single most common way candidates fail this interview, even with strong technical knowledge
- How to convert "500M daily active users" into an actual QPS estimate, step by step

Prerequisites: Stages 1–8 (this is the synthesis layer). **Real-world transfer:** This is literally the interview format at most companies that ask system design.

STAGE 10 — Case Studies (apply Stage 9's method every single time)

Each one run as a mock interview: I play interviewer, you drive, I interrupt the way a real interviewer would ("why not X instead?").

General (roughly increasing difficulty):

1. URL Shortener
2. Rate Limiter (as a full system, not just the algorithm from Stage 6)
3. Chat Application (WhatsApp-style)
4. News Feed (Twitter/Facebook-style)
5. (*stretch*) Distributed Job Scheduler / Web Crawler

ML-Specific: 6. Recommendation System (YouTube/TikTok-style) — direct extension of Stage 6 of your ML roadmap 7. Real-Time Fraud Detection — extension of your ML roadmap's fraud project, now at production scale with latency constraints 8. Model Serving Pipeline at Scale 9. LLM-Powered Application (RAG system, chat product, or agent platform) — extension of your ML roadmap's Stage 8 GPT-from-scratch work into a production architecture

Must be able to do by the end: Take a system design prompt you've never seen, and run the full Stage 9 method to a defensible, tradeoff-justified design — without recalling a memorized solution.

Suggested Pacing

Stage	Estimated time (part-time)
0	~1 day
1	~1 week
2	~1 week
3	~2 weeks (core — don't rush)
4	~4–5 days
5	~2 weeks
6	~1 week
7	~4–5 days
8	~2–3 weeks
9	ongoing, revisit before every case study
10	ongoing, ~1–2 case studies/week once foundations are solid

Total: roughly 8–10 weeks of foundations, then ongoing case-study practice.

How this maps to your goal

By Stage 6 you can reason about most "classic" system design interview questions with real justification instead of pattern-matched answers. By Stage 8 you can extend that into ML-serving- specific system design, which is the differentiated skill set given your ML background. Stage 10 is where it all gets stress-tested against novel prompts — the actual target skill you described.